ROOT@NOTES:~$ FIELD GUIDE · A PRACTICAL SYNTAX GUIDE

BASH SCRIPTING

This guide is a reference for writing and reading Bash (`.sh`) scripts. It explains the language in small pieces, then ends with examples that combine those pieces. The examples are deliberately fragments rather than complete script skeletons.

26 sections · Bash 3.2 – 5.4 · typeset 20 Sep 2026 from bash-scripting-guide.md

BASH | SHELL | SYNTAX REFERENCE | NOT SH

KEY IDEA

Bash is not the same as `sh`. Features such as `[[ ... ]]`, arrays, `(( ... ))`, process substitution, `mapfile`, and `declare` are Bash-specific. If a script uses them, run it with Bash rather than assuming `/bin/sh` supports them.

BEFORE YOU BEGIN: BASH VERSIONS MATTER

Check the interpreter you will actually use:

```
bash --version
/usr/bin/env bash --version
```

Some syntax in this guide requires a newer Bash release:

BASH VERSION	FEATURES FROM THIS GUIDE AVAILABLE BY THAT VERSION
3.2	Indexed arrays, `[[ ... ]]`, `=~`, arithmetic, and process substitution
4.0+	Associative arrays, `mapfile`, `coproc`, `globstar`, `&>>`, `|&`, and `^^` / `,,` case conversion
4.2+	The `[[ -v variable ]]` existence test
4.3+	Name references with `declare -n`
4.4+	`${value@Q}`-style transformations and `mapfile -d`

> ● CONTEXT
>
> The system-provided Bash on many Macs is 3.2, so examples using the later features require a separately installed newer Bash. A script always gets the version selected by its shebang or invocation; your interactive shell version does not change that.

[1] RUNNING BASH CODE

1.1 The shebang

The first line can tell the operating system which interpreter to use when the file is executed directly:

```
#!/usr/bin/env bash
```

`env` searches `PATH` for Bash. A fixed path is more predictable when the environment is controlled:

```
#!/bin/bash
```

The shebang matters when running `./script.sh`. It is ignored when explicitly running `bash script.sh`.

Make a file executable with:

```
chmod +x script.sh
./script.sh
```

Other useful ways to invoke Bash:

```
bash script.sh            # Run a file in a new Bash process
bash -n script.sh         # Check syntax without executing commands
bash -x script.sh         # Trace commands as they execute
bash -c 'printf "%s\n" "$HOME"'  # Run a command string
```

Sourcing runs a file in the **current** shell, so variable and directory changes remain afterward:

```
source settings.sh
. settings.sh             # Equivalent POSIX spelling
```

[2] COMMANDS, WHITESPACE, AND COMMENTS

A simple command consists of a command name and zero or more arguments:

```
printf '%s\n' "hello world"
```

Spaces and tabs normally separate words. A newline ends a command. A semicolon can separate commands on one line:

```
name=Ada; printf 'Hello, %s\n' "$name"
```

A backslash immediately followed by a newline continues the same logical line:

```
printf '%s %s\n' \
  "first" \
  "second"
```

Comments begin with `#` when it appears where a new word could begin:

```
# Whole-line comment
count=3  # End-of-line comment
printf '%s\n' "# this is data, not a comment"
```

Bash has no dedicated block-comment syntax. A common way to temporarily ignore literal text is a no-op command with a quoted here-document:

```
: <<'COMMENT'
Nothing in this block is executed or expanded.
$HOME remains literal here.
COMMENT
```

`:` is a builtin that does nothing successfully.

[3] QUOTING AND ESCAPING

Quoting controls how Bash interprets special characters.

3.1 Unquoted text

Unquoted expansions may undergo word splitting and filename expansion. Literal glob characters are also active:

```
printf '%s\n' *.txt
```

3.2 Single quotes

Single quotes preserve every character literally. A single quote cannot occur inside single quotes:

```
printf '%s\n' '$HOME is not expanded'
printf '%s\n' 'can'"'"'t'           # Adjacent quoted pieces form one word
```

3.3 Double quotes

Double quotes preserve whitespace and glob characters while still allowing parameter expansion, command substitution, and arithmetic expansion:

```
name='Grace Hopper'
printf 'Hello, %s\n' "$name"
printf 'Home: %s; files: %s\n' "$HOME" "$(find . -type f | wc -l)"
```

Inside double quotes, backslash only has special meaning before `$`, backtick, `"`, `\`, or a newline.

3.4 Backslash

A backslash quotes the next character when unquoted:

```
printf '%s\n' one\ word
printf '%s\n' \$HOME
```

3.5 ANSI-C quoting

`$'...'` understands C-style escapes:

```
printf '%s' $'first\nsecond\tcolumn\n'
```

Common escapes include `\n`, `\t`, `\r`, `\\`, `\'`, `\xHH`, and `\uHHHH`.

3.6 Locale-aware quoting

`$"..."` marks a string for locale translation. It is uncommon in ordinary scripts and should not be confused with `$'...'`.

3.7 The default rule

Quote expansions unless you specifically want splitting or globbing:

```
rm -- "$file"             # One exact pathname
printf '%s\n' "$value"    # Preserve whitespace and empty values
```

The `--` convention tells many commands that later words are operands, not options.

[4] VARIABLES AND ENVIRONMENT VARIABLES

Assignments have no spaces around `=`:

```
name='Ada Lovelace'
count=5
empty=
```

Read a variable with parameter expansion:

```
printf '%s\n' "$name"
printf '%s\n' "${name} wrote notes"
```

Braces remove ambiguity:

```
item=book
printf '%s\n' "${item}s"   # books, not the variable $items
```

Shell variables belong to the current shell. Exported variables are copied into child-process environments:

```
export APP_MODE=development
export PATH="$HOME/bin:$PATH"
```

An assignment before a command changes that command's environment only:

```
LC_ALL=C sort names.txt
DEBUG=1 ./program
```

Remove a variable with `unset`:

```
unset name
unset 'scores[2]'
```

4.1 Variable attributes

`declare` creates variables with Bash-specific attributes:

```
declare -i total=0          # Integer: assignments are evaluated arithmetically
declare -r constant='fixed' # Read-only
declare -x mode='test'      # Exported
declare -a items            # Indexed array
declare -A lookup           # Associative array
declare -n ref=target       # Name reference to another variable
```

Inside a function, use `local` to avoid changing a caller's variable:

```
local result
local -i index=0
```

Useful introspection:

```
declare -p name             # Reusable representation of a variable
printf '%q\n' "$name"      # Shell-escaped representation of a value
```

[5] SHELL EXPANSIONS

Understanding expansions explains most surprising Bash behavior. In broad terms, Bash performs:

1. brace expansion;
2. tilde expansion;
3. parameter, arithmetic, and command substitution (and process substitution);
4. word splitting;
5. filename expansion;
6. quote removal.

The exact rules depend on context. Quoting normally suppresses word splitting and filename expansion.

5.1 Brace expansion

Brace expansion generates text before variables are expanded:

```
printf '%s\n' file{1,2,3}.txt
printf '%s\n' {01..05}
printf '%s\n' {a..f..2}    # a, c, e
mkdir -p project/{src,test,docs}
```

Because it happens first, variables do not define brace ranges:

```
end=3
printf '%s\n' {1..$end}    # Does not produce 1 2 3
```

5.2 Tilde expansion

At the start of an unquoted word, `~` represents a home directory:

```
printf '%s\n' ~            # Current user's home
printf '%s\n' ~otheruser   # Another user's home, if known
```

Prefer `"$HOME/path"` when building a path inside quotes; `"~/path"` does not expand the tilde.

5.3 Parameter expansion

Basic forms:

```
$name
${name}
${array[index]}
```

Defaults and required values:

```
${value:-fallback}          # Use fallback if unset or empty
${value-fallback}           # Use fallback only if unset
${value:=fallback}          # Assign fallback if unset or empty, then use it
${value=fallback}           # Assign only if unset
${value:?error message}     # Print error and abort if unset or empty
${value?error message}      # Error only if unset
${value:+replacement}       # Use replacement if set and nonempty
${value+replacement}        # Use replacement if set
```

The colon makes “empty” count the same as “unset.”

String length and slicing:

```
text='abcdef'
printf '%s\n' "${#text}"       # 6
printf '%s\n' "${text:2:3}"    # cde
printf '%s\n' "${text: -2}"    # ef; space distinguishes it from :-
```

Prefix and suffix removal use glob patterns:

```
path='/var/log/app.log'
printf '%s\n' "${path#*/}"     # var/log/app.log: shortest matching prefix
printf '%s\n' "${path##*/}"    # app.log: longest matching prefix
printf '%s\n' "${path%.*}"     # /var/log/app: shortest matching suffix
printf '%s\n' "${path%%/*}"    # Empty here: longest matching suffix
```

Replacement and case conversion:

```
value='one two two'
printf '%s\n' "${value/two/2}"   # one 2 two
printf '%s\n' "${value//two/2}"  # one 2 2
printf '%s\n' "${value/#one/1}"  # Replace at beginning
printf '%s\n' "${value/%two/2}"  # Replace at end
printf '%s\n' "${value^^}"       # Uppercase all characters
printf '%s\n' "${value,,}"       # Lowercase all characters
```

Pattern-based case change can use one character (`^` or `,`) or all matching characters (`^^` or `,,`).

Transformations include:

```
printf '%s\n' "${value@Q}"     # Quoted for reuse as shell input
printf '%s\n' "${value@A}"     # Assignment/declaration representation
```

Indirect expansion treats a value as another variable's name:

```
target='answer'
answer=42
printf '%s\n' "${!target}"     # 42
```

For extensive indirect access, name references (`declare -n`) are usually clearer.

5.4 Command substitution

`$(...)` runs commands in a subshell and substitutes their standard output, removing trailing newlines:

```
current_dir=$(pwd)
line_count=$(wc -l < data.txt)
printf '%s\n' "Today is $(date +%F)"
```

The older backtick form works but nests and quotes poorly:

```
result=`command`            # Valid, but prefer $(command)
```

> • RULE OF THUMB
>
> Quote command substitutions unless splitting is intentional.

5.5 Arithmetic expansion

Arithmetic expansion substitutes an integer result:

```
x=7
printf '%d\n' "$((x * 3 + 1))"
next=$((x + 1))
```

Variable names inside arithmetic expressions do not need `$` . Bash integer arithmetic supports common C operators, including `+ - * / % **` , comparisons, bitwise operators, `&&` , `||` , `?:` , assignments, and pre/post increment. A leading `0` may mean octal; force base 10 with `10#` :

```
month='08'
number=$((10#$month))
```

Division is integer division, and dividing by zero is an error.

5.6 Word splitting and IFS

After an **unquoted** parameter, command, or arithmetic expansion, Bash splits results using characters in `IFS` (space, tab, and newline by default):

```
words='one two three'
printf '<%s>\n' $words      # Three arguments
printf '<%s>\n' "$words"    # One argument
```

Use `read` when intentionally splitting structured text:

```
IFS=: read -r user _ uid _ _ home shell < /etc/passwd
```

> **RULE OF THUMB**
>
> Avoid changing global `IFS` unnecessarily.

5.7 Filename expansion (globbing)

Unquoted patterns expand to matching pathnames:

```
*.txt                   # Any name ending in .txt
file?.log               # Exactly one character at ?
[abc]*                  # Begins with a, b, or c
[!0-9]*                 # Does not begin with a digit
```

Useful glob-related shell options:

```
shopt -s nullglob       # An unmatched glob expands to nothing
shopt -s failglob       # An unmatched glob is an error
shopt -s dotglob        # Globs can match leading-dot names
shopt -s globstar       # ** recursively traverses directories
shopt -s extglob        # Enable extended patterns
```

Extended glob patterns are:

```
?(pattern-list)            # Zero or one occurrence
*(pattern-list)            # Zero or more
+(pattern-list)            # One or more
@(pattern-list)            # Exactly one
!(pattern-list)            # Anything except the pattern
```

Alternatives inside a pattern list use `|`, such as `@(jpg|png)`.

[6] EXIT STATUS AND COMMAND CHAINING

Every command returns an integer status. `0` means success; `1`–`255` mean some kind of failure.

```
command
status=$?                  # Status of the most recent command
printf 'status=%d\n' "$status"
```

Prefer testing a command directly instead of inspecting `$?` later:

```
if mkdir archive; then
  printf '%s\n' 'directory created'
fi
```

Logical lists use exit statuses:

```
compile && printf '%s\n' 'build succeeded'
compile || printf '%s\n' 'build failed' >&2
first && second || fallback
```

`A && B` runs `B` only if `A` succeeds. `A || B` runs `B` only if `A` fails. Do not treat `A && B || C` as a perfect ternary expression: if `B` fails, `C` also runs.

A semicolon or newline runs the next command regardless:

```
first; second
```

An ampersand runs the preceding pipeline asynchronously:

```
long_job &
job_pid=$!
wait "$job_pid"
```

[7] TEST EXPRESSIONS

7.1 [[...]] for Bash conditions

Inside `[[ ... ]]`, words do not undergo ordinary word splitting or filename expansion. This makes it the preferred Bash conditional construct.

String tests:

```
[[ -z $text ]]             # Empty string
[[ -n $text ]]             # Nonempty string
[[ $left == "$right" ]]    # Exact equality
[[ $left != "$right" ]]    # Inequality
[[ $left < $right ]]       # Lexicographic order
[[ $name == *.txt ]]       # Pattern match; pattern is deliberately unquoted
```

Quoting the right side of `==` makes it literal:

```
[[ $name == "*.txt" ]]     # Matches the literal text *.txt
```

Regular expression matching uses `=~`:

```
if [[ $line =~ ^([A-Za-z]+):([0-9]+)$ ]]; then
  label=${BASH_REMATCH[1]}
  number=${BASH_REMATCH[2]}
fi
```

Store complex regular expressions in a variable to simplify quoting:

```
pattern='^[[:space:]]*#'
[[ $line =~ $pattern ]]
```

Integer comparisons:

```
[[ $a -eq $b ]]            # Equal
[[ $a -ne $b ]]            # Not equal
[[ $a -lt $b ]]            # Less than
[[ $a -le $b ]]            # Less than or equal
[[ $a -gt $b ]]            # Greater than
[[ $a -ge $b ]]            # Greater than or equal
```

File tests:

```
[[ -e $path ]]             # Exists
[[ -f $path ]]             # Regular file
[[ -d $path ]]             # Directory
[[ -L $path ]]             # Symbolic link
[[ -r $path ]]             # Readable
[[ -w $path ]]             # Writable
[[ -x $path ]]             # Executable/searchable
[[ -s $path ]]             # Exists and has nonzero size
[[ $a -nt $b ]]            # a is newer than b
[[ $a -ot $b ]]            # a is older than b
[[ $a -ef $b ]]            # Same device and inode
```

Combine tests with `&&`, `||`, `!`, and parentheses:

```
[[ -f $path && ( -r $path || $force == yes ) ]]
```

7.2 `[ ... ]` and `test`

`[` is a command, so spaces around its arguments and closing `]` are required:

```
if [ -f "$path" ]; then
  printf '%s\n' 'regular file'
fi

test -f "$path"            # Same underlying test
```

Quote expansions inside `[ ... ]`. For portable `sh`, combine commands outside the brackets:

```
[ -f "$path" ] && [ -r "$path" ]
```

7.3 ((...)) for arithmetic conditions

Arithmetic evaluation succeeds when its result is nonzero:

```
if (( count > 10 && count % 2 == 0 )); then
  printf '%s\n' 'large even count'
fi

((count++))                # Increment; status is based on the old value
((++count))                # Increment; status is based on the new value
```

> • TRAP
>
> Under `set -e`, standalone arithmetic whose result is zero can unexpectedly terminate a script. Use an assignment such as `count=$((count + 1))` when status is irrelevant.

[8] CONDITIONAL CONSTRUCTS

8.1 if, elif, and else

`if` checks the exit status of a command or command list:

```
if [[ -d $path ]]; then
  printf '%s\n' 'directory'
elif [[ -f $path ]]; then
  printf '%s\n' 'regular file'
else
  printf '%s\n' 'something else or missing'
fi
```

The condition can be any command:

```
if grep -q 'needle' file.txt; then
  printf '%s\n' 'found it'
fi
```

8.2 case

`case` compares a word against glob patterns:

```
case $answer in
  y|yes|YES)
    printf '%s\n' 'accepted'
    ;;
  n|no|NO)
    printf '%s\n' 'declined'
    ;;
  *)
    printf 'Unknown answer: %s\n' "$answer" >&2
    ;;
esac
```

Case terminators:

- `;;` stops after the matching branch.
- `;&` continues with the next branch's commands without testing its pattern.
- `;;&` continues testing later patterns.

[9] LOOPS

9.1 for over words or array elements

```
for color in red green blue; do
  printf '%s\n' "$color"
done

for file in ./*.txt; do
  [[ -e $file ]] || continue
  printf '%s\n' "$file"
done

for arg; do                # Equivalent to: for arg in "$@"
  printf 'argument: %s\n' "$arg"
done
```

TRAP

Do not parse filenames with `for file in $(find ...)`; command substitution loses boundaries. Use globs, `find ... -exec`, or NUL-delimited input instead.

9.2 C-style arithmetic for

```
for ((i = 0; i < 5; i++)); do
  printf '%d\n' "$i"
done
```

Any of the three expressions may be omitted.

9.3 while

```
count=3
while ((count > 0)); do
  printf '%d\n' "$count"
  ((--count))
done
```

Read a file without damaging whitespace or backslashes:

```
while IFS= read -r line || [[ -n $line ]]; do
  printf 'line: %s\n' "$line"
done < input.txt
```

The `|| [[ -n $line ]]` portion processes a final line that has no newline terminator.

9.4 until

`until` repeats while its condition fails:

```
until [[ -e ready.flag ]]; do
  sleep 1
done
```

9.5 Loop control

```
continue                  # Skip to the next iteration
continue 2                # Affect two nested loop levels
break                     # Leave the loop
break 2                   # Leave two nested loop levels
```

[10] FUNCTIONS

The recommended function syntax is:

```
greet() {
  local name=${1:-world}
  printf 'Hello, %s\n' "$name"
}
```

You may also encounter the Bash-specific `function` keyword:

```
function greet {
  printf 'Hello, %s\n' "${1:-world}"
}
```

`function greet()` is accepted by Bash too, but mixing both forms adds nothing. The `name() { ...; }` form is clearer and is also understood by POSIX shells.

Call it like any command:

```
greet 'Linus'
```

Functions receive positional parameters of their own. Variables are otherwise global unless declared `local`.

`return` sets the function's exit status; it does **not** return an arbitrary string:

```
is_even() {
  (( $1 % 2 == 0 ))
}

if is_even 12; then
  printf '%s\n' 'even'
fi
```

To produce data, write it to standard output and capture it:

```
double() {
  printf '%d\n' "$(( $1 * 2 ))"
}

result=$(double 21)
```

> • RULE OF THUMB
>
> Keep diagnostic messages on standard error so they do not contaminate captured output.

[11] SCRIPT AND FUNCTION ARGUMENTS

Special parameters:

```
$0                      # Script or shell name
$1, $2, ...             # Positional arguments
${10}                   # Braces required for 10 and above
$#                      # Number of positional arguments
"$@"                    # Every argument as a separate word
"$*"                    # All arguments joined into one word by first IFS char
$?                      # Most recent exit status
$$                      # Current shell's process ID
$!                      # PID of most recent background pipeline
$-                      # Current shell option flags
$_                      # Last argument of the previous simple command (context-dependent)
```

Almost always iterate arguments with `"$@"`:

```
for argument in "$@"; do
  printf '<%s>\n' "$argument"
done
```

Remove positional arguments with `shift`:

```
first=$1
shift
printf 'Remaining count: %d\n' "$#"
```

Replace them with `set --`:

```
set -- alpha 'two words' gamma
```

11.1 Parsing short options with getopts

`getopts` handles short options without launching another program:

```
while getopts ':f:n:v' option; do
  case $option in
    f) file=$OPTARG ;;
    n) count=$OPTARG ;;
    v) verbose=true ;;
    :) printf 'Option -%s needs a value\n' "$OPTARG" >&2 ;;
    \?) printf 'Unknown option: -%s\n' "$OPTARG" >&2 ;;
  esac
done
shift "$((OPTIND - 1))"
```

In `:f:n:v`, a colon after a letter means that option needs an argument. A leading colon enables controlled error handling. Remaining non-option operands are in `"$@"` after the `shift`.

[12] INDEXED ARRAYS

Create and access an indexed array:

```
colors=(red 'light green' blue)
colors[3]=yellow
printf '%s\n' "${colors[1]}"
```

Expand elements safely:

```
printf '%s\n' "${colors[@]}"  # Each element remains a separate word
printf '%s\n' "${colors[*]}"  # All elements joined into one word
printf '%d\n' "${#colors[@]}" # Number of elements
printf '%s\n' "${!colors[@]}" # Existing indices
```

Slices and appending:

```
subset=("${colors[@]:1:2}")
colors+=(purple 'dark orange')
```

Arrays can be sparse, so `${#array[@]}` is not necessarily the next free index. Append with `+=` rather than calculating an index.

Iterate safely:

```
for color in "${colors[@]}"; do
  printf '<%s>\n' "$color"
done
```

[13] ASSOCIATIVE ARRAYS

Associative arrays use string keys and must be declared:

```
declare -A capitals=(
  [Canada]=Ottawa
  [Japan]=Tokyo
  [France]=Paris
)

capitals[Italy]=Rome
printf '%s\n' "${capitals[Japan]}"
```

Iterate keys and values:

```
for country in "${!capitals[@]}"; do
  printf '%s -> %s\n' "$country" "${capitals[$country]}"
done
```

Associative-array iteration order is not guaranteed. Check whether a key exists with:

```
if [[ -v 'capitals[Canada]' ]]; then
  printf '%s\n' 'key exists'
fi
```

[14] INPUT AND OUTPUT

14.1 printf

Prefer `printf` for predictable output:

```
printf '%s\n' "$message"
printf 'name=%s count=%d\n' "$name" "$count"
printf -v padded '%05d' 42  # Assign output to a variable
```

Useful format conversions include `%s` (string), `%d` (integer), `%f` (floating-point formatting), `%q` (shell-escaped), and `%b` (interpret backslash escapes in an argument).

> TRAP
>
> Never let untrusted data become the format string:
>
> ```
> printf '%s\n' "$user_text"   # Safe
> ```

14.2 echo

`echo` is convenient for simple fixed strings, but handling of `-n` and backslash escapes varies. Prefer `printf` in reusable scripts.

14.3 read

```
IFS= read -r line           # Preserve leading/trailing whitespace and backslashes
read -r first rest          # Split across variables using IFS
read -r -p 'Name: ' name    # Prompt (Bash-specific)
read -r -s -p 'Secret: ' secret  # Silent input
read -r -n 1 key            # One character
read -r -t 5 answer         # Five-second timeout
read -r -u 3 line           # Read from file descriptor 3
```

14.4 mapfile / readarray

Read lines directly into an indexed array:

```
mapfile -t lines < input.txt
mapfile -d '' -t files < <(find . -type f -print0)
```

`-t` removes the delimiter; `-d ''` uses NUL as the delimiter. `readarray` is a synonym.

[15] REDIRECTION AND FILE DESCRIPTORS

Every process normally begins with three file descriptors:

- `0`: standard input (`stdin`)
- `1`: standard output (`stdout`)
- `2`: standard error (`stderr`)

Basic redirections:

```
command > output.txt        # Replace file with stdout
command >> output.txt       # Append stdout
command < input.txt         # Read stdin from file
command 2> errors.txt       # Replace file with stderr
command 2>> errors.txt      # Append stderr
command > all.txt 2>&1      # Send stdout and stderr to the same destination
command &> all.txt          # Bash shorthand for both streams
command &>> all.txt         # Bash shorthand to append both streams
command > /dev/null         # Discard stdout
command 2>&1                # Send stderr wherever stdout currently goes
```

Order matters because redirections are applied left to right:

```
command > file 2>&1         # Both streams go to file
command 2>&1 > file         # stderr goes to old stdout; stdout goes to file
```

Other descriptor forms:

```
command 3< input.txt        # Open descriptor 3 for reading
command 3> output.txt       # Open descriptor 3 for writing
command 3>> output.txt      # Open descriptor 3 for appending
command 3>&1                # Duplicate stdout onto descriptor 3
command 3<&0                # Duplicate stdin onto descriptor 3
command 3>&-                # Close output descriptor 3
command 3<&-                # Close input descriptor 3
command <> data.txt         # Open the file on stdin for reading and writing
```

`set -C` (also called `set -o noclobber`) prevents `>` from replacing an existing regular file. `>|` overrides that setting for one redirection:

```
set -C
command >| deliberately-replaced.txt
set +C
```

Attach a redirection to a whole compound command:

```
{
  printf '%s\n' 'first'
  printf '%s\n' 'second'
} > output.txt
```

15.1 Here-documents

A here-document supplies multiple lines of input:

```
cat <<EOF
User: $USER
Directory: $PWD
EOF
```

Quote the delimiter to prevent expansion:

```
cat <<'EOF'
$USER remains literal.
So does $(date).
EOF
```

`<<-EOF` strips leading **tab characters** from the body and delimiter, which can help indentation.

15.2 Here-strings

A here-string supplies one string plus a trailing newline:

```
grep -q 'needle' <<< "$text"
```

15.3 Managing custom descriptors

Let Bash allocate an unused descriptor:

```
exec {log_fd}>> application.log
printf '%s\n' 'started' >&"$log_fd"
exec {log_fd}>&-            # Close it
```

`exec` with only redirections changes the current shell's descriptors.

[16] PIPELINES

A pipeline connects each command's stdout to the next command's stdin:

```
producer | filter | consumer
```

By default, a pipeline's status is the last command's status. Enable `pipefail` to use the rightmost failing status instead:

```
set -o pipefail
generate | transform | save
printf '%s\n' "${PIPESTATUS[@]}"  # Status of every command in last foreground pipeline
```

`|&` pipes both stdout and stderr from the left command:

```
command |& logger
```

Pipeline commands normally run in subshell environments, so variable assignments made inside may not survive:

```
count=0
printf '%s\n' a b | while read -r _; do
  ((count++))
done
printf '%d\n' "$count"      # Commonly still 0
```

Redirect the loop instead when its state must remain:

```
count=0
while read -r _; do
  ((count++))
done < <(printf '%s\n' a b)
```

[17] PROCESS SUBSTITUTION

Process substitution exposes a command's input or output as a file-like path:

```
diff <(sort old.txt) <(sort new.txt)
tee >(gzip > copy.gz) >(sha256sum > copy.sha256) < source.dat > /dev/null
```

- `<(command)` lets another command read `command`'s output.
- `>(command)` lets another command write into `command`'s input.

Unlike command substitution, process substitution does not capture output into a string and therefore does not strip trailing newlines.

[18] GROUPING, SUBSHELLS, AND COMMAND SCOPE

Braces run commands in the current shell. Spaces and a terminating semicolon or newline are required:

```
{
  value=changed
  printf '%s\n' "$value"
}
printf '%s\n' "$value"      # changed
```

Parentheses run commands in a subshell:

```
location=$PWD
(
  cd /tmp || exit
  printf '%s\n' "$PWD"
)
printf '%s\n' "$PWD"        # Original directory
```

Subshells are useful for temporary directory, option, trap, or variable changes.

[19] COMMAND LOOKUP AND EXECUTION

Bash can run:

- reserved words such as `if`, `for`, and `case`;
- aliases;
- functions;
- builtins such as `cd`, `printf`, and `read`;
- executable files found through `PATH`.

Inspect command resolution with:

```
type -a printf
command -V python
```

Bypass a function or alias and use ordinary command lookup with:

```
command printf '%s\n' 'text'
```

Run an external file directly with its path. `exec command ...` replaces the current shell process rather than creating another one:

```
exec server --port 8080
```

[20] JOBS, PROCESSES, AND SIGNALS

Background commands and PIDs:

```
worker &
pid=$!

if wait "$pid"; then
  printf '%s\n' 'worker succeeded'
else
  printf '%s\n' 'worker failed' >&2
fi
```

Useful job-control commands in interactive shells include `jobs`, `fg`, and `bg`. Non-interactive scripts normally track PIDs and use `wait`.

Send a signal with `kill`:

```
kill -TERM "$pid"
kill -0 "$pid"             # Check whether signaling is possible; sends no signal
```

20.1 Traps

`trap` registers commands for signals or shell events:

```
temporary_file=$(mktemp)
trap 'rm -f -- "$temporary_file"' EXIT
```

Common traps include `EXIT`, `ERR`, `DEBUG`, `RETURN`, `INT`, `TERM`, and `HUP`.

```
trap 'printf "%s\n" "interrupted" >&2; exit 130' INT
trap - INT                 # Restore default handling
trap '' HUP                # Ignore a signal
trap -p                    # Display current traps
```

TRAP

Be careful with quoting: a single-quoted trap action is evaluated when the trap fires, while expansions in a double-quoted action happen when the trap is installed.

[21] SHELL OPTIONS AND ERROR HANDLING

Options controlled by `set` include:

```
set -e                    # errexit: exit after certain unhandled failures
set -u                    # nounset: error on many unset-variable expansions
set -o pipefail           # Pipeline fails if any component fails
set -x                    # Trace expanded commands
set -n                    # Parse without executing
set +x                    # Disable tracing
```

The often-used combination is:

```
set -Eeuo pipefail
```

NOT MAGIC

It is useful, but not magic:

- `-e` has exceptions in conditions, logical lists, pipelines, and other contexts.
- `-u` requires care with optional variables and empty arrays, especially on older Bash versions.
- `pipefail` changes which pipelines count as failures.
- `-E` makes `ERR` traps inherit into functions, substitutions, and subshells.

Handle expected failures explicitly:

```
if ! output=$(possibly_failing_command); then
  printf '%s\n' 'command failed' >&2
fi
```

`shopt` controls Bash-specific behaviors:

```
shopt -s nullglob
shopt -u nullglob
shopt -q nullglob && printf '%s\n' 'enabled'
```

Temporarily trace a suspicious region:

```
set -x
value=$(transform "$input")
set +x
```

> TRAP
>
> Do not enable tracing around secrets: expanded command arguments are printed.

[22] COMMON BUILTINS AND KEYWORDS

These appear frequently in scripts:

```
cd directory                # Change working directory
pwd                         # Print working directory
pushd directory             # Push directory and change to it
popd                        # Return to pushed directory
printf ...                  # Formatted output
read ...                    # Read fields or a line
mapfile ...                 # Read lines into an array
source file                 # Execute a file in this shell
alias name='command'        # Textual command alias (mainly interactive use)
unalias name                # Remove alias
declare ...                 # Set variable attributes
local ...                   # Function-local variable
readonly name               # Make a variable immutable
export name                 # Add variable to child environments
unset name                  # Remove variable or function
type command                # Describe command resolution
help builtin                # Show Bash help
history                     # Shell history (mainly interactive)
hash                        # Manage remembered command paths
true                        # Successful status
false                       # Failing status
:                            # Successful no-op
return [status]             # Leave function or sourced file
exit [status]               # Leave the shell
break [n]                   # Leave loop(s)
continue [n]                # Advance loop(s)
wait [pid]                  # Wait for background work
trap action signal          # Install signal/event handler
umask mask                  # Set permissions mask for newly created files
```

Use `help name` for Bash's built-in documentation, for example `help read`, `help [[`, or `help declare`.

[23] LESS-COMMON SYNTAX WORTH RECOGNIZING

23.1 Coprocesses

`coproc` starts an asynchronous process with pipes for two-way communication:

```
coproc TRANSFORMER { tr '[:lower:]' '[:upper:]'; }
printf '%s\n' 'hello' >&"${TRANSFORMER[1]}"
exec {TRANSFORMER[1]}>&-
IFS= read -r reply <&"${TRANSFORMER[0]}"
wait "$TRANSFORMER_PID"
```

23.2 select

`select` displays a numbered menu and reads a choice:

```
select color in red green blue; do
  [[ -n $color ]] || continue
  printf 'Selected %s\n' "$color"
  break
done
```

The raw user input is available in `REPLY`.

23.3 Time a pipeline

The `time` reserved word can time a whole pipeline:

```
time generate | compress > archive.gz
```

Customize its Bash output with the `TIMEFORMAT` variable.

23.4 Negating a pipeline

`!` reverses a pipeline's success status:

```
if ! grep -q 'enabled' config.ini; then
  printf '%s\n' 'setting is absent'
fi
```

[24] SAFE PATTERNS AND COMMON MISTAKES

24.1 Quote data, not syntax

```
[[ $file == *.txt ]]        # Pattern is syntax, so leave it unquoted
rm -- "$file"               # File value is data, so quote it
```

24.2 Preserve arguments with arrays

Do not assemble a command in a string:

```
# Fragile: options='--name "two words"'
#          program $options
```

Use an array:

```
options=(--name 'two words')
[[ $verbose == true ]] && options+=(--verbose)
program "${options[@]}"
```

24.3 Avoid eval for ordinary data

`eval` parses text as shell code, which easily creates quoting bugs and command-injection vulnerabilities. Arrays, associative arrays, `printf -v`, and name references solve most cases without reparsing code.

24.4 Keep filenames intact

Filenames can contain spaces, tabs, newlines, glob characters, and leading hyphens. Quote expansions and prefer NUL-delimited interfaces:

```
while IFS= read -r -d '' file; do
  printf '%q\n' "$file"
done < <(find . -type f -print0)
```

24.5 Do not parse ls

Use globs or `find` rather than treating display-oriented `ls` output as machine-readable input.

24.6 Check cd

```
cd "$directory" || exit 1
```

Otherwise later commands may run in the wrong directory.

24.7 Use mktemp for temporary paths

```
temp_dir=$(mktemp -d) || exit 1
trap 'rm -rf -- "$temp_dir"' EXIT
```

> ● TRAP
>
> Only remove a path you created and validated. Never construct a broad removal target from unchecked or possibly empty variables.

24.8 Prefer $(...) over backticks

It nests cleanly and makes quoting easier to read.

24.9 Know when glob and regex syntax differ

```
[[ $name == *.txt ]]        # Glob
[[ $name =~ \.txt$ ]]       # Extended regular expression
```

`*` means "any string" in a glob but "repeat the previous item" in a regular expression.

24.10 Use ShellCheck

ShellCheck detects many quoting, portability, and data-flow mistakes:

```
shellcheck script.sh
```

[25] COMBINED EXAMPLES

These fragments deliberately omit boilerplate. Read them by tracing expansions, conditions, exit statuses, and data flow.

25.1 Example A: Build a command safely

```
input='records with spaces.csv'
output='selected.csv'
column=4
verbose=true

awk_options=(-F, -v "column=$column")
[[ $verbose == true ]] && awk_options+=(-v verbose=1)

awk "${awk_options[@]}" '
  NR == 1 || $column != "" {
    if (verbose) printf "keeping line %d\n", NR > "/dev/stderr"
    print
  }
' "$input" > "$output"
```

THINGS TO NOTICE

- The array preserves each option as one argument.
- `[[ ... ]] && ...` conditionally appends an option.
- The single-quoted AWK program is passed literally to `awk`; its `$column` belongs to AWK, not Bash.
- The quoted Bash pathnames remain single arguments.
- Standard output becomes the result file while AWK diagnostics go to standard error.

25.2 Example B: Classify files and count extensions

```
declare -A extension_counts=()

while IFS= read -r -d '' path; do
  name=${path##*/}

  if [[ $name == *.* && $name != .* ]]; then
    extension=${name##*.}
    ((extension_counts[$extension] += 1))
  else
    ((extension_counts['(none)'] += 1))
  fi
done < <(find . -maxdepth 1 -type f -print0)

for extension in "${!extension_counts[@]}"; do
  printf '%-12s %d\n' "$extension" "${extension_counts[$extension]}"
done
```

• THINGS TO NOTICE

› `find -print0` and `read -d ''` preserve every valid filename.

› Process substitution keeps the `while` loop in the current shell, so its array updates survive.

› `${path##*/}` and `${name##*.}` remove the longest matching prefixes.

› The associative array maps each extension to its count.

25.3 Example C: Validate and transform structured input

```
record_pattern='^([[:alpha:]_][[:alnum:]_]*):([0-9]+)$'

while IFS= read -r record || [[ -n $record ]]; do
  [[ -z $record || $record == \#* ]] && continue

  if [[ $record =~ $record_pattern ]]; then
    key=${BASH_REMATCH[1]}
    raw_number=${BASH_REMATCH[2]}
    number=$((10#$raw_number))
    printf '%s=%d\n' "${key^^}" "$((number * 2))"
  else
    printf 'invalid record: %q\n' "$record" >&2
  fi
done < records.txt
```

THINGS TO NOTICE

- Blank lines and lines beginning with `#` are skipped.
- The regular expression captures a key and decimal digits.
- `10#` prevents leading-zero values from being interpreted as octal.
- `${key^^}` uppercases the key; arithmetic expansion doubles the value.
- `%q` prints invalid input in an unambiguous escaped form.

25.4 Example D: Run concurrent work and collect failures

```
declare -A task_for_pid=()
pids=()

for task in alpha beta gamma; do
  process_one "$task" > "${task}.out" 2> "${task}.err" &
  pid=$!
  pids+=("$pid")
  task_for_pid[$pid]=$task
done

failed=0
for pid in "${pids[@]}"; do
  if wait "$pid"; then
    printf 'finished: %s\n' "${task_for_pid[$pid]}"
  else
    printf 'failed: %s\n' "${task_for_pid[$pid]}" >&2
    failed=1
  fi
done

((failed == 0))
```

THINGS TO NOTICE

- `&` starts each function call concurrently and `$!` captures its PID.
- An indexed array retains launch order; an associative array relates each PID to a task.
- `wait` returns the corresponding job's status.
- The last arithmetic condition produces an overall success or failure status.

25.5 Example E: Use a temporary workspace with cleanup

```
work_dir=$(mktemp -d) || exit 1
trap 'rm -rf -- "$work_dir"' EXIT
trap 'exit 130' INT
trap 'exit 143' TERM

if curl -fsS "$url" -o "$work_dir/download" \
  && gzip -t "$work_dir/download"; then
  gzip -dc "$work_dir/download" > "$destination"
else
  printf 'download or validation failed\n' >&2
  exit 1
fi
```

THINGS TO NOTICE

- The temporary directory is created before the cleanup trap is installed.
- The `EXIT` trap runs for normal completion and most explicit exits.
- `&&` prevents validation unless the download succeeds.
- The `if` condition treats failure as expected control flow.
- Quoted paths protect the generated pathname as well as the destination.

25.6 Example F: Compare two generated views without intermediate files

```
if diff -u \
  <(cut -d: -f1 /etc/passwd | sort) \
  <(printf '%s\n' "${expected_users[@]}" | sort); then
  printf '%s\n' 'user lists match'
else
  status=$?
  ((status == 1)) \
    && printf '%s\n' 'user lists differ' >&2 \
    || printf 'diff failed with status %d\n' "$status" >&2
fi
```

THINGS TO NOTICE

- Each `<(...)` becomes a readable file-like input for `diff`.
- `"${expected_users[@]}"` passes every array item separately to `printf`.
- `diff` uses status `1` for "different" and values above `1` for an operational error.
- The multiline logical list distinguishes those meanings.

[26] READING UNFAMILIAR BASH

When studying a script, use this order:

1. Identify how it is invoked and which Bash options are enabled.
2. Find inputs: arguments, environment variables, files, and stdin.
3. Mark expansions and decide which are quoted.
4. Track each command's stdout, stderr, redirections, and exit status.
5. Note subshell boundaries: pipelines, `(...)`, command substitutions, and process substitutions.
6. Track mutations to variables, arrays, the working directory, traps, and background jobs.
7. Inspect external commands with their own documentation; Bash only arranges their arguments and streams.

Useful investigation commands:

```
bash -n script.sh           # Syntax check
shellcheck script.sh        # Static analysis
bash -x script.sh           # Execution trace
help '[['                   # Bash help for conditional syntax
help read                   # Bash help for the read builtin
type -a command_name        # What Bash will actually run
```

THE WHOLE THING

With the syntax above, most Bash scripts become a combination of four ideas: construct exact arguments, route byte streams, branch on exit statuses, and keep track of which shell process owns each piece of state.

- Source — bash-scripting-guide.md
- Static analysis — `shellcheck script.sh`
- Built-in help — `help read` , `help '[['` , `help declare`

Every callout colour means the same thing on every page: green is the thing to remember, cyan is context, amber is a rule of thumb, red is a trap, lime is a worked example.